

Composição e o inventário

O personagem ganha uma mochila, os itens ganham comportamento próprio, e você descobre a outra metade da orientação a objetos: a que não é herança.

Continuação direta da parte 2.

Você precisa do jogo fechado, com `_vida`, `curar` e o Dragão funcionando antes de começar.

ANTES DE COMEÇAR

Onde você está

Na parte 1 você aprendeu herança: uma classe **é um** tipo de outra. Na parte 2 você aprendeu encapsulamento: cada objeto cuida do próprio estado.

Agora vem a última peça, e ela é a que falta para o seu RPG virar um jogo de verdade: **composição**. Um objeto guarda outro objeto dentro de si.

A teoria desta parte está em `teoria-poo/parte-c-composicao/`. Se um exercício não fizer sentido, leia a seção com o mesmo nome lá.

A sua pasta

Até aqui você tinha três arquivos. Nesta parte entra **um só**, e ele se chama `itens.py`. Nenhum outro.

```
rpg/  
  personagem.py    a classe Personagem  
  classes.py       Guerreiro, Mago, Goblin, Orc, Troll, Dragao  
  itens.py         NOVO: Item, PocaoDeVida, Bomba, Inventario  
  main.py          o menu e o combate
```

A REGRA CONTINUA VALENDO

Quatro arquivos. Não existe `itens2.py`, não existe `item.py` no singular, não existe `inventario.py` separado. Se você já tem arquivo sobrando na pasta, apague antes de começar. O código pronto de cada parte está em `gabarito/`.

O caminho

1 O problema

Você faz a poção herdar de Personagem e vê o resultado absurdo.

2 A classe Item

Um item guarda nome e sabe ser usado.

3 Dois itens, um comando

Poção e bomba, mesma chamada, efeitos opostos.

4 A mochila

Uma classe que é dona de uma lista.

5 Ligar ao personagem

Composição de verdade, e o `usar_pocao` morre.

6 Duas ferramentas do Python

`__str__` e `@property`.

7 Fechar o jogo

Inimigo derrotado deixa um item para trás.

Lembrete: os extras do anexo B continuam opcionais. Pular todos não te atrapalha em nada.

Sumário

01	O problema: a poção que ataca	5
02	A classe Item	7
03	Dois itens, um comando	9
04	A mochila	12
05	Ligar ao personagem	14
06	Duas ferramentas do Python	17
07	Fechar o jogo	20
AN	Anexo A: erros que você vai ver	22
AN	Anexo B: extras	24
AN	Anexo C: checklist de entrega	26

CAPÍTULO 1

O problema: a poção que ataca

Antes de aprender a forma certa, faça a errada. Este capítulo existe para você sentir o incômodo.

Você acabou de passar duas partes aprendendo herança. Então, para criar uma poção, a primeira ideia é herdar de `Personagem`.

Exercício 1: a poção que herda

ARQUIVO: CLASSES.PY · TEMPORÁRIO

```
rpg/  
  personagem.py    nao mexa  
  classes.py       <-- voce esta aqui  
  main.py
```

Faça:

1. No fim do `classes.py`, crie a classe `Pocao`, herdando de `Personagem`.
2. No `__init__` dela, chame a mãe com: `"Poção de vida", 1, 0, 0, 0`.
3. Crie uma poção e um Goblin.
4. Mande a poção atacar o Goblin. Depois mande a poção se defender. Depois pergunte se a poção está viva.

O QUE VOCÊ DEVE VER

Tudo funciona. Nenhum erro na tela. A poção ataca, se defende e está viva.

Esse é o problema. O Python não te avisa de nada. O programa está errado por dentro e continua rodando.

Responda antes de seguir: uma poção precisa de `ataque`? Precisa de `defesa`? Precisa de uma mochila dentro dela?

A PERGUNTA QUE RESOLVE

Diga em voz alta: **"Poção é um Personagem?"** Se a frase soar estranha, herança está errada. A frase certa aqui é outra: **"Personagem tem poções."**

Apague a classe `Pocao` **agora.** Ela não vai ser usada. A partir do capítulo 2 você escreve a versão certa, e ela não herda de `Personagem`.

Terminou o capítulo 1? Ele não tem extras. Os extras começam no checkpoint 1.

CAPÍTULO 2

A classe Item

Um item não é um personagem. Um item é uma coisa que tem nome e que sabe ser usada. Isso é uma classe nova, do zero, sem mãe nenhuma.

Exercício 2: criar o arquivo e a classe base

ARQUIVO: ITENS.PY · CRIE AGORA

```
rpg/  
  personagem.py    nao mexa  
  classes.py  
  itens.py         <-- CRIE este arquivo  
  main.py
```

Faça:

1. Crie o arquivo `itens.py`, na mesma pasta dos outros três.
2. Dentro dele, crie a classe `Item`. Ela **não herda de nada**. A linha é só `class Item:`.
3. Dê a ela um `__init__` que recebe `self` e `nome`, e guarda o nome.
4. Dê a ela um método `usar` que recebe `self`, `dono` e `inimigo`. Por enquanto ele só imprime que o item não faz nada.

Por que `usar` recebe os dois? Alguns itens afetam quem usou (a poção). Outros afetam o adversário (a bomba). Se todos recebem os dois, cada item escolhe sozinho quem ele mexe, e quem chama não precisa saber a diferença. Você vai ver isso funcionar no capítulo 3.

CHECKPOINT 1

Existe o arquivo `itens.py`, e ele tem a classe `Item`.

`Item` não herda de `Personagem` nem de nada.

A classe `Pocao` do capítulo 1 foi apagada do `classes.py`.

A sua pasta tem exatamente quatro arquivos.

TESTE DO CHECKPOINT 1

```
from itens import Item

i = Item("Pedra")
print(i.nome)
print(i.atacar)
```

Resultado esperado: aparece `Pedra`, e depois um erro.

```
AttributeError: 'Item' object has no attribute 'atacar'
```

O erro é o sucesso. Um item não ataca. Compare com o capítulo 1, onde a poção atacava sem reclamar.

Terminou o checkpoint 1? Extras: anexo B, item E1.1.

CAPÍTULO 3

Dois itens, um comando

Agora os itens de verdade. Os dois herdam de `Item`, e desta vez a frase "é um" é verdadeira: poção **é um** item, bomba **é um** item.

Exercício 3: a poção

ARQUIVO: ITENS.PY

```
rpg/  
  personagem.py    nao mexa  
  classes.py  
  itens.py         <-- voce esta aqui  
  main.py
```

Faça:

1. Crie a classe `PocaoDeVida`, herdando de `Item`.
2. No `__init__` dela, sem parâmetro nenhum além do `self`, chame a mãe com o nome `"Poção de vida"`.
3. Depois disso, guarde `self.cura` valendo `25`.
4. Sobrescreva o `usar`. Ele deve mandar o **dono** se curar, usando o método `curar` que você escreveu na parte 2.
5. Imprima o que aconteceu.

NÃO FAÇA A CONTA AQUI

O item **não** mexe em `dono._vida`. Ele chama `dono.curar(...)` e deixa o personagem aplicar as regras dele. É o encapsulamento da parte 2 valendo entre objetos diferentes.

Exercício 4: a bomba

ARQUIVO: ITENS.PY

Faça:

1. Crie a classe `Bomba`, herdando de `Item`, com o nome `"Bomba"` e `self.dano` valendo `30`.
2. Sobrescreva o `usar`. Ele deve mandar o **inimigo** receber dano, usando `receber_dano`.

3. Repare: o `usar` da bomba tem **exatamente os mesmos parâmetros** do `usar` da poção. Só o corpo muda.

CHECKPOINT 2

`PocaoDeVida` e `Bomba` herdam de `Item`.

Os dois `usar` recebem `self`, `dono` e `inimigo`.

A poção mexe só no dono. A bomba mexe só no inimigo.

Nenhum dos dois toca em `_vida` diretamente.

TESTE DO CHECKPOINT 2

```
from classes import Guerreiro, Goblin
from itens import PocaoDeVida, Bomba

heroi = Guerreiro("Thoric")
heroi._vida = 50
bicho = Goblin()

for item in [PocaoDeVida(), Bomba()]:
    item.usar(heroi, bicho)

print("heroi:", heroi.vida, "| goblin:", bicho.vida)
```

Resultado esperado: o herói sobe para 75 e o Goblin cai para 12.

Olhe o laço. Ele chama `item.usar(heroi, bicho)` sem perguntar que item é. Não tem `if`. É o mesmo polimorfismo da parte 1, agora em objetos que não são personagens.

Se o herói não subiu para 75: a sua poção provavelmente está mexendo na vida direto em vez de chamar `curar`.

LEIA A FONTE

Documentação oficial do Python, em português, seção **9.5 Herança**:

docs.python.org/pt-br/3/tutorial/classes.html

Você usou herança em `Item` e composição no `Personagem`. A documentação fala das duas. Escreva com suas palavras por que `PocaoDeVida` herda de `Item`, mas `Inventario` não vai herdar de `Personagem`:

Terminou o checkpoint 2? Extras: anexo B, itens E2.1 e E2.2.

A mochila

Você poderia guardar os itens numa lista solta dentro do personagem. Vai funcionar. Mas aí a regra de "qual índice é o item 1" e "o que acontece se o número não existe" fica espalhada por todo lugar que abre a mochila.

Uma classe resolve isso: a lista ganha dono.

Exercício 5: a classe Inventario

ARQUIVO: ITENS.PY

```
rpg/  
  personagem.py    nao mexa  
  classes.py  
  itens.py         <-- voce esta aqui  
  main.py
```

Faça:

1. No mesmo `itens.py`, crie a classe `Inventario`. Ela **não herda de nada**.
2. No `__init__`, crie `self._itens` como uma lista vazia. Com underscore: a lista é assunto interno.
3. Crie `adicionar(self, item)`, que põe o item na lista.
4. Crie `quantidade(self)`, que devolve quantos itens tem.
5. Crie `esta_vazio(self)`, que devolve verdadeiro ou falso.
6. Crie `listar(self)`, que imprime cada item numerado a partir de **1**, não de zero.
7. Crie `tirar(self, numero)`. Ele converte o número para índice, devolve `None` se o número não existir, e senão **remove e devolve** o item.

Por que `tirar` devolve `None` em vez de quebrar? Porque quem chama é o menu, e o jogador vai digitar 9 numa mochila de 2 itens. Tratar isso aqui, uma vez, é melhor do que lembrar de tratar em todo lugar.

CHECKPOINT 3

Existe a classe `Inventario`, e ela não herda de nada.

A lista se chama `_itens`, com underscore.

`tirar` remove o item e devolve ele.

`tirar` com número inválido devolve `None` e não quebra.

TESTE DO CHECKPOINT 3

```
from itens import Inventario, PocaoDeVida, Bomba

mochila = Inventario()
print(mochila.esta_vazio())

mochila.adicionar(PocaoDeVida())
mochila.adicionar(Bomba())
mochila.listar()

print(mochila.tirar(99))
print(mochila.quantidade())
```

Resultado esperado:

```
True
1 - <itens.PocaoDeVida object at 0x...>
2 - <itens.Bomba object at 0x...>
None
2
```

Sim, a listagem está feia. Aquele `<itens.PocaoDeVida object at 0x...>` é o Python dizendo "não sei como transformar isto em texto". **O capítulo 6 conserta.** Por enquanto, o que importa é que o `tirar(99)` devolveu `None` e a quantidade continuou 2.

Terminou o checkpoint 3? Extras: anexo B, item E3.1.

CAPÍTULO 5

Ligar ao personagem

Agora a composição de verdade. O personagem vai **ter uma** mochila.

Exercício 6: o personagem ganha a mochila

ARQUIVO: PERSONAGEM.PY · AGORA VOCÊ MEXE NELE

```
rpg/  
  personagem.py    <-- voce esta aqui  
  classes.py  
  itens.py  
  main.py
```

Faça:

1. No topo do `personagem.py`, importe `Inventario` e `PocaoDeVida` do arquivo `itens`.
2. No `__init__`, **apague** a linha `self.pocoes = pocoes`.
3. No lugar dela, crie `self.inventario` valendo um `Inventario()` novo.
4. Logo abaixo, use um `for` que repete `pocoes` vezes e adiciona uma `PocaoDeVida()` na mochila a cada volta.
5. No `mostrar_status`, troque a linha das poções por uma que mostre `self.inventario.quantidade()`.

REPRE NO QUE VOCÊ NÃO PRECISOU FAZER

O parâmetro `pocoes` continua existindo no `__init__`. Por isso **nenhuma** classe filha mudou: Guerreiro, Mago, Goblin, Orc, Troll e Dragão continuam chamando `super().__init__(...)` exatamente igual. Você trocou o interior do `Personagem` e as seis filhas não ficaram sabendo.

Exercício 7: o menu da mochila

ARQUIVO: MAIN.PY

```
rpg/
  personagem.py
  classes.py
  itens.py
  main.py          <-- voce esta aqui
```

Faça:

1. No `turno_do_jogador`, troque o texto da opção 3 para `"3 - Abrir a mochila"`.
2. Faça a opção 3 chamar uma função nova, `abrir_mochila(jogador, inimigo)`.
3. Escreva essa função. Se a mochila estiver vazia, avise e saia.
4. Senão, liste os itens e peça um número, avisando que `0` volta.
5. Tire o item escolhido da mochila. Se vier `None`, avise que o item não existe e saia.
6. Se vier um item, mande ele ser usado, passando o jogador e o inimigo.

Exercício 8: apagar o `usar_pocao`

ARQUIVO: `PERSONAGEM.PY`

O `usar_pocao` não tem mais função. A mochila faz o trabalho dele, e faz melhor: agora serve para qualquer item, não só poção.

Faça:

1. Apague o método `usar_pocao` inteiro do `personagem.py`.
2. Rode o jogo e confirme que nada quebrou.

CHECKPOINT 4

O `Personagem` tem um `self.inventario`.

Nenhuma classe do `classes.py` precisou mudar.

A opção 3 do menu abre a mochila e usa um item.

O método `usar_pocao` não existe mais.

O `main.py` nunca escreve `_itens`.

TESTE DO CHECKPOINT 4

```
from classes import Guerreiro, Mago, Goblin

print(Guerreiro("Thoric").inventario.quantidade())
print(Mago("Elric").inventario.quantidade())
print(Goblin().inventario.quantidade())
```

Resultado esperado: 2, 4 e 0. São exatamente os números que já estavam no `super().__init__(...)` de cada classe.

Se der `TypeError` sobre argumentos: você tirou o parâmetro `pocoes` do `__init__`. Ele tem que continuar lá. Quem mudou foi o que você faz com ele.

Se der `AttributeError: 'NoneType'`: você declarou `self.inventario` mas esqueceu de escrever `Inventario()` com os parênteses.

Terminou o checkpoint 4? Extras: anexo B, itens E4.1 e E4.2.

CAPÍTULO 6

Duas ferramentas do Python

Este capítulo não muda a estrutura do jogo. Ele arruma duas coisas feias que sobraram, e as duas são ferramentas que você vai usar para sempre.

Exercício 9: `__str__`, ou como o objeto vira texto

ARQUIVO: ITENS.PY, DEPOIS PERSONAGEM.PY

Lembra do `<itens.PocaoDeVida object at 0x...>` do capítulo 4? Aquilo é o Python dizendo que não sabe como mostrar o seu objeto.

Existe um método especial para responder isso. Ele se chama `__str__`, com dois underscores de cada lado, igual ao `__init__`. Você nunca chama ele na mão: o `print` chama por você.

Faça:

1. Na classe `Item`, crie o método `__str__`, que recebe só `self` e **devolve** o nome do item. Use `return`, não `print`.
2. Rode o teste do checkpoint 3 de novo e veja a listagem ficar legível.
3. Na classe `Personagem`, crie um `__str__` que devolve o nome com a vida atual e a máxima, no formato `Thoric (120/120)`.

Por que `return` e não `print`? Porque `__str__` responde "qual é o seu texto", e quem decide o que fazer com esse texto é quem perguntou. Se você usar `print` dentro dele, a função devolve `None` e o Python reclama.

Exercício 10: `@property`, a porta de leitura

ARQUIVO: PERSONAGEM.PY

Na parte 2 você fechou a porta trocando `self.vida` por `self._vida`. Isso resolveu o `inimigo.vida = -999`, mas cobrou um preço: hoje nem **ler** a vida funciona de fora.

Ler nunca foi o problema. O problema era escrever.

Faça:

1. Na classe `Personagem`, escreva a linha `@property` sozinha.

- Logo abaixo dela, crie um método chamado `vida`, que recebe só `self` e devolve `self._vida`.
- Faça o mesmo para `vida_maxima`.

CHECKPOINT 5

`Item` tem `__str__`, e a mochila lista nomes legíveis.

`Personagem` tem `__str__`.

`personagem.vida` pode ser lido de fora.

`personagem.vida = 999` continua proibido.

TESTE DO CHECKPOINT 5

```
from classes import Guerreiro
from itens import PocaDeVida

heroi = Guerreiro("Thoric")
print(heroi)
print(PocaDeVida())
print(heroi.vida)
heroi.vida = 999
```

Resultado esperado:

```
Thoric (120/120)
Poção de vida
120
AttributeError: property 'vida' of 'Guerreiro' object has no setter
```

Os três primeiros mostram, o quarto quebra. É exatamente isso que você queria: ler liberado, escrever bloqueado.

Se o print mostrar `<classes.Guerreiro object at ...>` : o seu `__str__` está com nome errado, ou ficou fora da classe. Confira a indentação e os dois underscores de cada lado.

LEIA A FONTE

Documentação oficial, na entrada da função **property**:

docs.python.org/pt-br/3/library/functions.html#property

A página mostra que uma property pode ter mais que leitura. Qual é o nome do que se escreve para permitir a **escrita** controlada?

E responda com suas palavras: por que a vida do personagem **não** deveria ganhar isso?

Terminou o checkpoint 5? Extras: anexo B, item E5.1.

Fechar o jogo

Falta uma coisa para a mochila fazer sentido: um jeito de ganhar itens durante o jogo. Senão você começa com duas poções e acabou.

Exercício 11: o inimigo derrotado deixa um item

ARQUIVO: MAIN.PY

```
rpg/  
  personagem.py  
  classes.py  
  itens.py  
  main.py      <-- voce esta aqui
```

Faça:

1. Importe `Bomba` e `PocaoDeVida` do `itens`.
2. No laço dos inimigos, logo depois do descanso de 50 de vida, escolha um prêmio: uma `Bomba()` se o inimigo derrotado foi o Troll, e uma `PocaoDeVida()` nos outros casos.
3. Adicione o prêmio à mochila do jogador.
4. Avise na tela o que caiu. Como o item tem `__str__`, você pode imprimir o objeto direto.

CHECKPOINT 6, O ÚLTIMO

O jogo roda do início ao fim: escolhe personagem, enfrenta os quatro inimigos, e termina com vitória ou derrota.

Dá para abrir a mochila no meio do combate e usar um item.

Cada inimigo derrotado deixa um item para trás.

Nenhum arquivo fora do `personagem.py` escreve `_vida`.

Nenhum arquivo fora do `itens.py` escreve `_itens`.

TESTE FINAL

Abra `main.py`, `classes.py` e `itens.py` e procure por `_vida` com Ctrl+F. Depois procure por `_itens` no `main.py` e no `personagem.py`.

Resultado esperado: nenhuma ocorrência em nenhum dos casos.

Se aparecer alguma, tem código de fora mexendo no que não é dele. Troque por uma chamada de método.

O que você construiu

CONCEITO	ONDE ESTÁ NO SEU JOGO
Classe e objeto	<code>Personagem</code> , <code>Item</code> , <code>Inventario</code> .
Herança	Guerreiro, Mago, Dragão, e também <code>PocaoDeVida</code> , <code>Bomba</code> .
Polimorfismo	<code>inimigo.atacar()</code> no combate, <code>item.usar()</code> na mochila.
Encapsulamento	<code>_vida</code> e <code>curar</code> ; <code>_itens</code> e <code>adicionar</code> .
Composição	<code>Personagem</code> tem um <code>Inventario</code> , que tem <code>Item</code> .

São os cinco conceitos da matéria, todos rodando ao mesmo tempo, num programa que você joga do começo ao fim.

ANEXO A

Erros que você vai ver

1. NONETYPE OBJECT HAS NO ATTRIBUTE

```
AttributeError: 'NoneType' object has no attribute 'adicionar'
```

Você escreveu `self.inventario = Inventario` sem os parênteses, ou declarou o atributo e nunca criou o objeto. Sem os parênteses você guardou a **classe**, não uma mochila.

ERRADO

```
self.inventario = Inventario
```

CERTO

```
self.inventario = Inventario()
```

2. IMPORTERROR OU CIRCULAR IMPORT

```
ImportError: cannot import name 'Inventario' from partially initialized module
```

O `personagem.py` importa do `itens.py`. Se você fizer o `itens.py` importar do `personagem.py`, os dois ficam se esperando.

Não precisa. O item recebe o personagem **como parâmetro** do `usar`, e nunca precisa importar a classe.

3. A LISTAGEM MOSTRA ENDEREÇOS DE MEMÓRIA

```
1 - <itens.PocaoDeVida object at 0x7f3c...>
```

Falta o `__str__`, ou ele está escrito errado. Confira: dois underscores de cada lado, e `return` em vez de `print`.

4. PROPERTY HAS NO SETTER

```
AttributeError: property 'vida' of 'Guerreiro' object has no setter
```

Depois do capítulo 6, isso pode ser certo ou errado.

É **certo** se você escreveu `personagem.vida = ...` de propósito, para testar.

É **erro** se aconteceu jogando. Nesse caso sobrou algum `self.vida = ...` dentro do `Personagem` que deveria ser `self._vida = ...`.

5. O ITEM SOME SEM SER USADO

Sem erro na tela: você abre a mochila, escolhe um item, e ele desaparece sem efeito nenhum.

O `tirar` já removeu o item da lista, mas você esqueceu de chamar o `usar` depois. Tirar e usar são duas ações.

ANEXO B

Extras

Tudo aqui é opcional. Nenhum capítulo depende destes itens.

E1.1 O ITEM QUE NÃO FAZ NADA

Depende de: *checkpoint 1*.

Crie um `Item("Pedra")` puro e use ele no combate. Ele vai imprimir que não faz nada.

Responda: por que faz sentido a classe base ter um `usar` que não faz nada, em vez de não ter `usar` nenhum?

E2.1 O ELIXIR

Depende de: *checkpoint 2*.

Crie um item que cura o dono e fere o inimigo, no mesmo `usar`. Confirme que o menu não precisou de nenhuma linha nova.

E2.2 A PEDRA DE AFIAR

Depende de: *checkpoint 2*.

Crie um item que aumenta o `ataque` do dono em 3, para sempre.

Pense antes: isso deveria passar por um método do personagem, como a cura passa pelo `curar`? Justifique a sua escolha.

E3.1 MOCHILA COM LIMITE

Depende de: *checkpoint 3*.

Faça a mochila aceitar no máximo 5 itens. O `adicionar` deve recusar o sexto e avisar.

Repare em quantas outras classes precisaram mudar para isso. Esse número é o ponto do exercício.

E4.1 O INIMIGO TAMBÉM USA ITENS

Depende de: *checkpoint 4*.

Dê poções a um inimigo e faça ele beber uma quando estiver abaixo de 30% da vida, em vez de atacar.

Onde essa decisão deve morar: no `main.py` ou na classe do inimigo? Justifique.

E4.2 MOCHILA CHEIA DE BOMBAS

Depende de: checkpoint 4.

Comece o jogo com três bombas além das poções. Jogue e decida se o jogo ficou fácil demais. Ajuste o dano ou a quantidade e explique a sua escolha.

E5.1 LEITURA DIFÍCIL

Depende de: checkpoint 5.

Este extra é de leitura. Abra a documentação dos métodos especiais:

docs.python.org/pt-br/3/reference/datamodel.html#object.__repr__

Existe `__str__` e existe `__repr__`. Responda: qual é a diferença de propósito entre os dois, e qual deles o Python usa quando você imprime uma **lista** de objetos?

ANEXO C

Checklist de entrega

FUNCIONA

- ☐ O jogo roda do início ao fim sem erro na tela.
- ☐ Dá para abrir a mochila durante o combate.
- ☐ Número inválido na mochila não quebra o jogo.
- ☐ Escolher 0 volta ao menu sem gastar o turno.
- ☐ Cada inimigo derrotado deixa um item.

COMPOSIÇÃO

- ☐ `Personagem` tem um `Inventario`.
- ☐ `Inventario` não herda de `Personagem`.
- ☐ `Item` não herda de `Personagem`.
- ☐ `PocaoDeVida` e `Bomba` herdam de `Item`.
- ☐ Nenhuma classe do `classes.py` precisou mudar no capítulo 5.

ENCAPSULAMENTO

- ☐ A lista da mochila se chama `_itens`.
- ☐ `main.py` e `personagem.py` não contêm `_itens`.
- ☐ O item de cura chama `curar`, e não mexe em `_vida`.
- ☐ O item de dano chama `receber_dano`.
- ☐ A vida é lida por `@property` e não pode ser escrita de fora.

ORGANIZAÇÃO

- ☐ Exatamente quatro arquivos na pasta.
- ☐ Nenhum arquivo com nome contendo espaço ou acento.
- ☐ O `usar_pocao` foi apagado.
- ☐ O código roda com `python3 main.py`, sem editar nada antes.

O ÚLTIMO TESTE

Acrescente um item novo, de qualquer tipo, ao seu jogo. Conte quantos arquivos você precisou abrir e quantos `if` precisou escrever fora do `itens.py`. Se a resposta for "um arquivo e nenhum `if`", você entendeu a matéria inteira.